

SECURITY AUDIT REPORT

Project: mx-chain-simulator-go-NewArc | Assessment Type: Real-World Threat Analysis

EXECUTIVE SUMMARY

This comprehensive security audit identifies 5 critical vulnerabilities within the mx-chain-simulator-go-NewArc codebase. These issues represent significant risks to the confidentiality, integrity, and availability of the simulator and its host environment. All findings are currently classified as **REJECTED**. The repository is deemed unfit for production or shared-environment deployment until the recommended remediations are verified across the target infrastructure.

#	VULNERABILITY TITLE	LOCATION	SEVERITY	STATUS
1	Unbounded Resource Consumption (DoS)	pkg/facade/simulatorFacade.go:L61	High	REJECT
2	Symlink Following in Config Fetching	pkg/proxy/configs/copy.go:L19	Medium	REJECT
3	Insecure Container Configuration (Root)	Dockerfile:L35	Medium	REJECT
4	Sensitive Data Persistence in /tmp	cmd/chainsimulator/main.go:L207	High	REJECT
5	Argument Injection in exec.Command	pkg/proxy/configs/configs.go:L40	Medium	REJECT

1. UNBOUNDED RESOURCE CONSUMPTION (DOS)

Severity: High | **CWE:** CWE-400 (Uncontrolled Resource Consumption)

Technical Description

The simulator's API provides endpoints to generate blocks or progress epochs. These functions wrap the core simulator logic and are protected by a global synchronization mutex. The implementation failed to validate integer inputs. An attacker can submit a request with an extremely large block count, forcing a massive CPU-bound loop while holding the global lock.

Operational Impact

A single malicious request will cause the simulator process to consume 100% CPU and block all other API requests indefinitely. This results in a total Denial of Service (DoS).

VULNERABLE CODE (BEFORE):

```
func (sf *simulatorFacade) GenerateBlocks(numOfBlocks int) error {  
  
    if numOfBlocks <= 0 {  
  
        return errInvalidNumOfBlocks  
  
    }  
  
    return sf.simulator.GenerateBlocks(numOfBlocks)
```

```
    }
```

REMEDIATED CODE (AFTER):

```
func (sf *simulatorFacade) GenerateBlocks(numOfBlocks int) error {

    if numOfBlocks <= 0 || numOfBlocks > maxBlocksToGenerate {

        return fmt.Errorf("invalid blocks count (max: %d)",
            maxBlocksToGenerate)

    }

    return sf.simulator.GenerateBlocks(numOfBlocks)

}
```

2. SYMLINK FOLLOWING IN CONFIG FETCHING

Severity: Medium | **CWE:** CWE-59 (Improper Link Resolution)

Technical Description

The `copyFolderWithAllFiles` function recursively traverses directories but follows symbolic links by default. If a repository contains a symlink pointing to a file outside the repository, the simulator will copy the contents of the target file into its workspace.

Operational Impact

This is an Arbitrary File Read vulnerability. Attackers can exfiltrate sensitive host-level files (SSH keys, credentials) by making them available within the simulator's workspace.

VULNERABLE CODE (BEFORE):

```
return filepath.Walk(src, func(path string, info os.FileInfo, err
error) error {

    if info.IsDir() {

        return os.MkdirAll(filepath.Join(dst, relPath), info.Mode())

    }

    in, err := os.Open(path)
```

```
■ // ... logic blindly copies file contents
```

```
■ })
```

■ REMEDIATED CODE (AFTER):

```
■ return filepath.Walk(src, func(path string, info os.FileInfo, err  
■ error) error {
```

```
■     if info.Mode()&os.ModeSymlink != 0 {
```

```
■         return nil // Securely skip symbolic links
```

```
■     }
```

```
■     if info.IsDir() {
```

```
■         return os.MkdirAll(filepath.Join(dst, relPath), info.Mode())
```

```
■     }
```

```
■ // ... rest of secure copy
```

```
■ })
```

3. INSECURE CONTAINER CONFIGURATION (ROOT)

■ **Severity:** Medium | **CWE:** CWE-250 (Execution with Unnecessary Privileges)

■ Technical Description

■ The `Dockerfile` does not define a non-root user, causing the simulator to run with UID 0 (root) inside the container. This violates the principle of least privilege.

■ Operational Impact

■ Running as root increases the blast radius of any exploit. It facilitates container escapes, unauthorized host volume access, and lateral movement within the network.

■ **VULNERABLE CODE (BEFORE):**

```
■ FROM ubuntu:22.04
```

```
■ COPY --from=builder /bin/chainsimulator /multiversx/
```

```
■ ENTRYPOINT ["/chainsimulator"]
```

■ REMEDIATED CODE (AFTER):

```
■ FROM ubuntu:22.04
```

```
■ RUN groupadd -r simulator && useradd -r -g simulator simulator
```

```
■ RUN chown -R simulator:simulator /multiversx
```

```
■ USER simulator
```

```
■ ENTRYPOINT ["/chainsimulator"]
```

■ 4. SENSITIVE DATA PERSISTENCE IN /TMP

■ **Severity:** High | **CWE:** CWE-312 (Cleartext Storage of Sensitive Information)

Technical Description

The simulator generates a `walletKey.pem` file in a temporary directory containing unencrypted private keys. The application lacks any logic to delete these files when the process terminates.

Operational Impact

On shared machines, these private keys remain on disk indefinitely. Any user with local access can recover these keys and gain control over the simulated blockchain authority.

VULNERABLE CODE (BEFORE):

```
func main() {  
  
    tempDir, _ := os.MkdirTemp(os.TempDir(), "sim-*")  
  
    startSimulator(tempDir)  
  
    // Application exits; tempDir remains on disk  
  
}
```

REMEDIATED CODE (AFTER):

```
func main() {  
  
    tempDir, _ := os.MkdirTemp(os.TempDir(), "sim-*")  
  
    defer os.RemoveAll(tempDir) // Atomic cleanup on exit  
  
    startSimulator(tempDir)  
  
}
```

5. ARGUMENT INJECTION IN EXEC.COMMAND

Severity: Medium | **CWE:** CWE-88 (Improper Neutralization of Argument Delimiters)

Technical Description

The application uses `exec.Command` with arguments derived from configuration. If these strings start with a hyphen, the binary interprets them as command-line flags.

Operational Impact

An attacker who can influence the configuration can achieve Remote Code Execution (RCE) by providing a repo URL that executes arbitrary payloads.

VULNERABLE CODE (BEFORE):

```
cmd := exec.Command("git", "clone", repoURL, destDir)

err := cmd.Run()
```

REMEDIATED CODE (AFTER):

```
// Use '--' to separate flags from positional arguments

cmd := exec.Command("git", "clone", "--", repoURL, destDir)

err := cmd.Run()
```